ARCHITECTURE & SYSTEM DESIGN

TableTalk

Intelligent Restaurant Assistant MCP Server

Complete High-Level Design (HLD), Low-Level Design (LLD), Sequence Flows, User Flows, and Agile User Story Specifications.

Model Context Protocol Spec: 2025-11-25 • Rust • Axum • SQLite WAL • React UI

Generated: October 2026

1. High-Level Design (HLD)

TableTalk connects voice interfaces (like Amazon Alexa+) and web applications to real restaurant operations via the Model Context Protocol over Streamable HTTP.

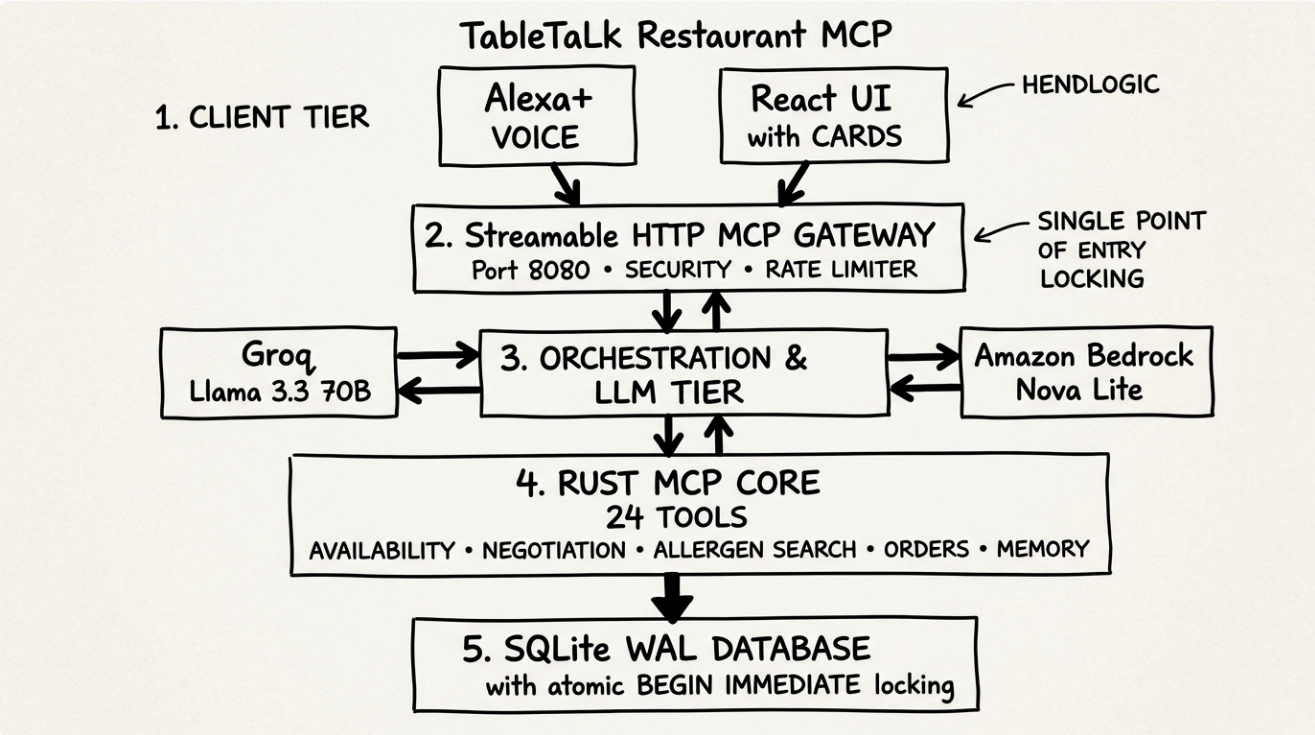


Figure 1: High-Level Architectural Tiers of the TableTalk Restaurant MCP Server

Architectural Tiers

- **Client Tier:** Alexa+ Voice Devices and React 19 Client rendering accessible UI cards.
- **Streamable HTTP MCP Gateway:** Port 8080 multiplexed endpoint with origin validation and rate limiting.
- **Conversational AI Tier:** Dynamic orchestration supporting Groq Llama 3.3 70B and Amazon Bedrock Nova Lite.
- **Rust MCP Core:** 24 strongly-typed tools managing dining availability, holds, allergens, and orders.
- **SQLite WAL Database:** Embedded persistence with atomic `BEGIN IMMEDIATE` concurrency serialization.

2. Low-Level Design (LLD)

The low-level design details component boundaries, request pipelines, concurrency controls, and persistence schemas.

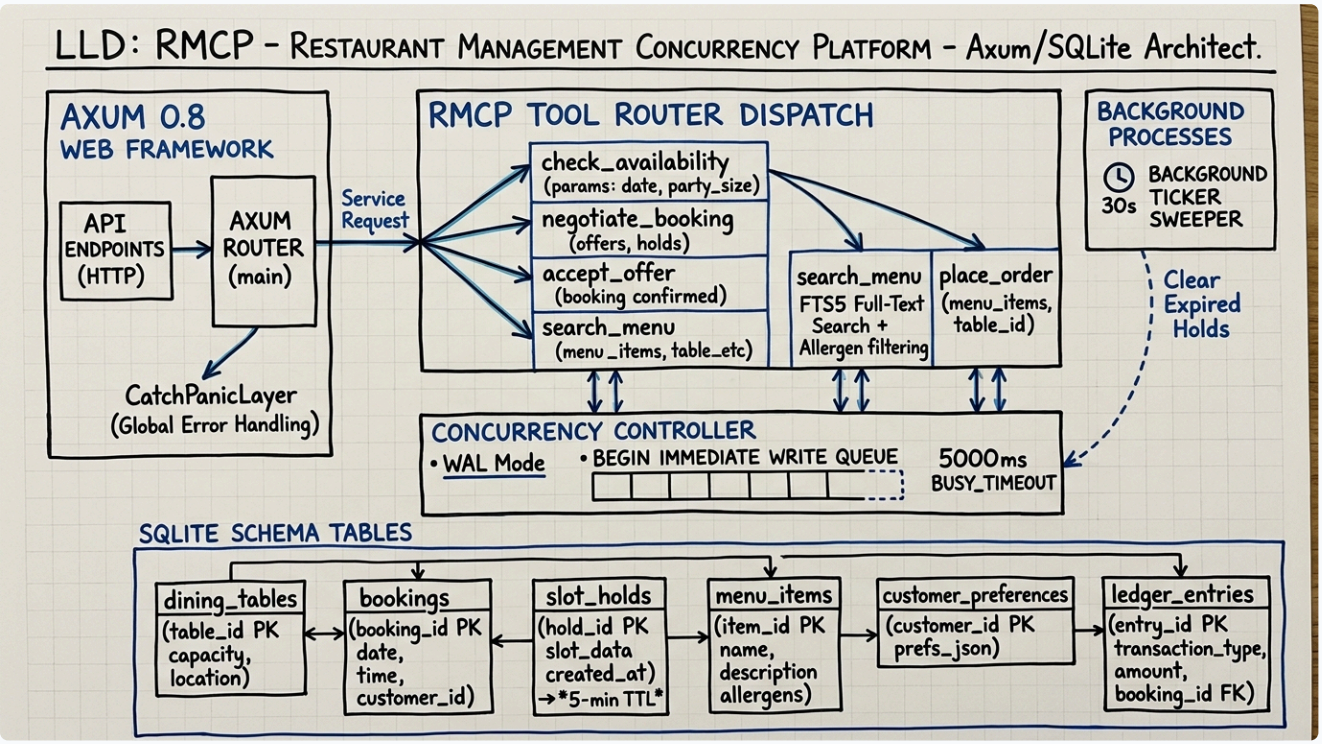


Figure 2: Component Architecture, RMCP Tool Router, and Concurrency Subsystem

Concurrency Controller

Uses SQLite WAL mode with `BEGIN IMMEDIATE` lock queueing and 5000ms busy timeout to prevent race conditions during table holds.

Background Sweeper

A Tokio interval ticker running every 30 seconds sweeps and releases expired 5-minute table holds back to the open pool.

FTS5 Allergen Filter

SQLite Full-Text Search virtual table synced with dietary indices, enforcing transparent cross-contact disclosures.

Integer Paise Money

All pricing, tax, and deposits are computed in minor units (paise/cents) to mathematically eliminate floating-point drift.

3. Sequence Diagram: Voice Negotiation Flow

Step-by-step trace showing the conversational capacity negotiation flow when a requested slot is occupied.

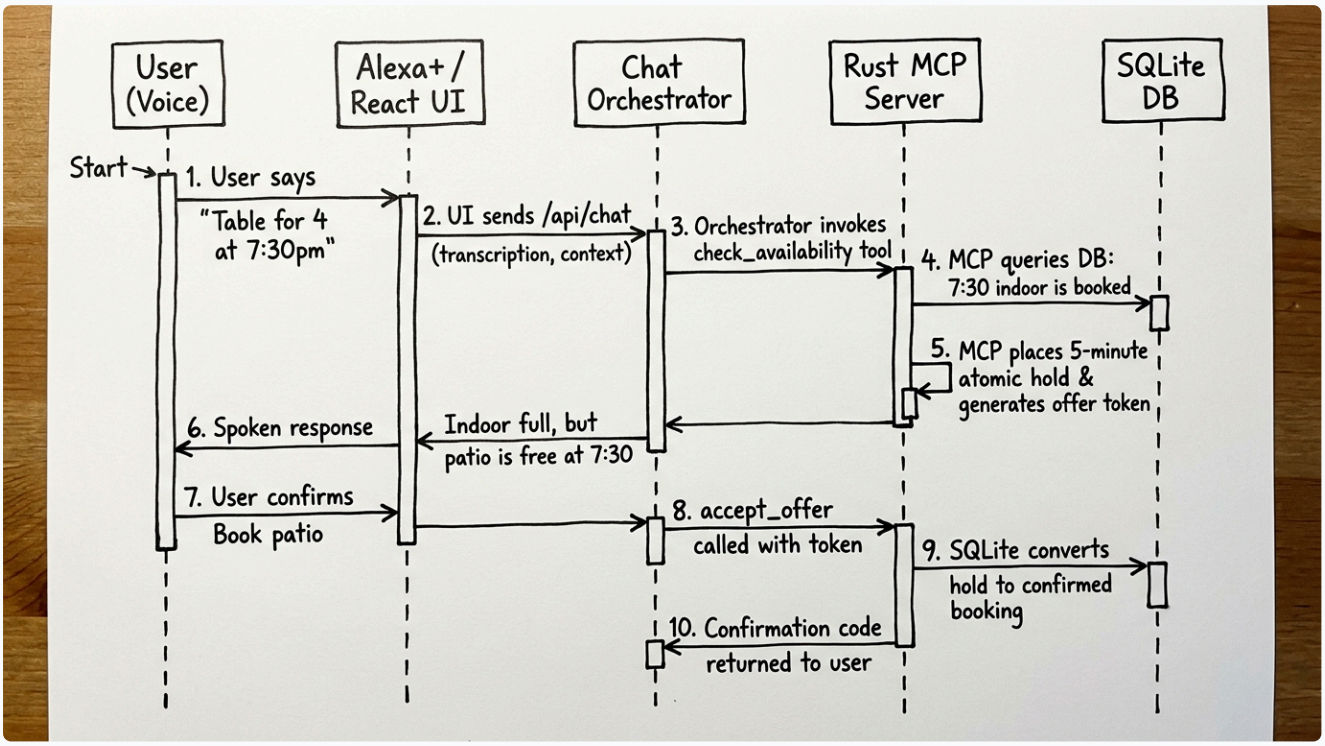


Figure 3: Multi-Turn Voice Negotiation, 5-Minute Atomic Hold, and Offer Acceptance Flow

Flow Highlights

- **Atomic Hold Placement:** When 7:30 PM indoor is fully booked, the server places a temporary 5-minute hold on the best ranked alternative (Patio at 7:30 PM).
- **Cryptographic Offer Tokens:** Offer tokens prevent table snatching by other users while the diner deliberates.
- **Idempotent Conversion:** Calling `accept_offer` converts the atomic hold into a confirmed booking.

4. User Flow: The Complete Dining Journey

Comprehensive flowchart tracing user journeys through search, negotiation, allergen safety, and ordering.

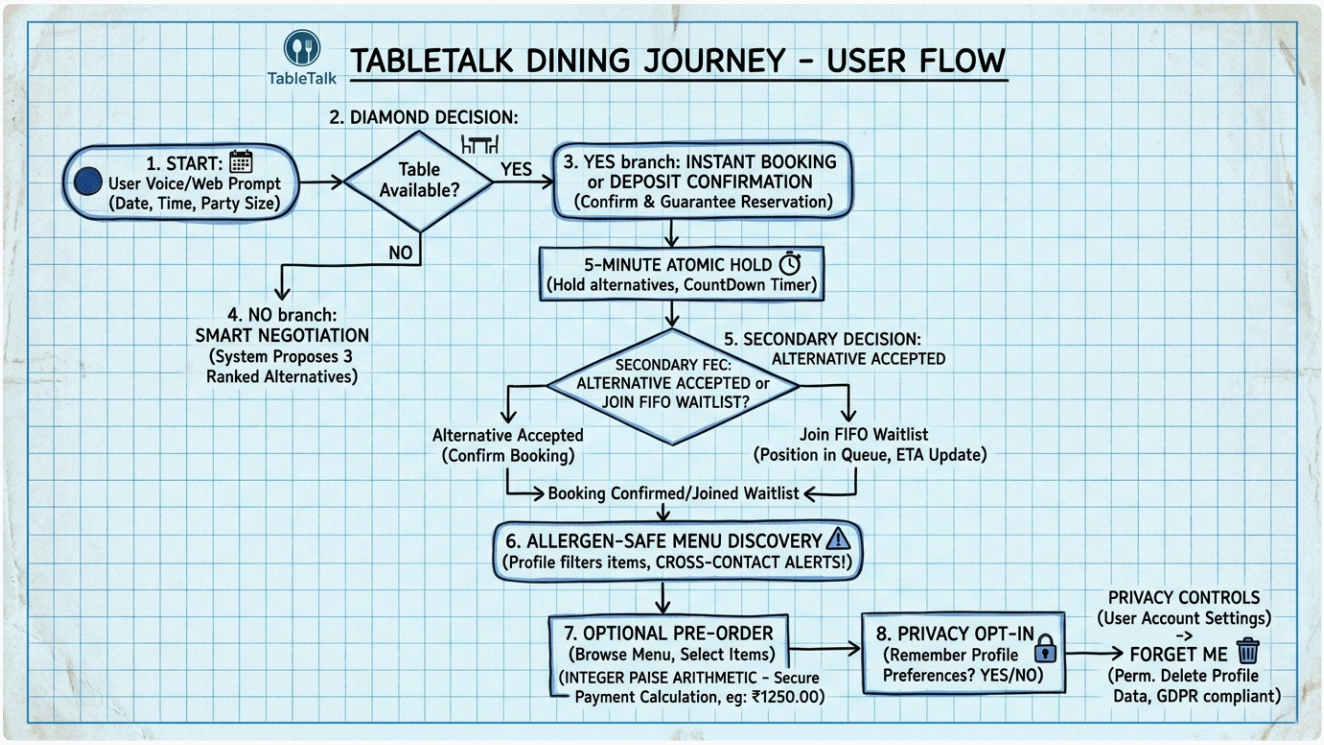


Figure 4: Complete Dining Lifecycle & Decision Trees

- **Smart Branching:** Immediate confirmation if available; ranked alternatives if occupied; FIFO waitlist if all slots full.
- **Allergen Safety Guardrails:** Prohibits false claims ("100% safe") and requires explicit confirmation.
- **Privacy Controls:** Preference memory requires explicit opt-in, backed by a permanent 1-click "Forget Me" erasure.

5. User Stories & Persona Journey Maps

Agile user stories capturing core persona needs, friction points, and acceptance criteria.

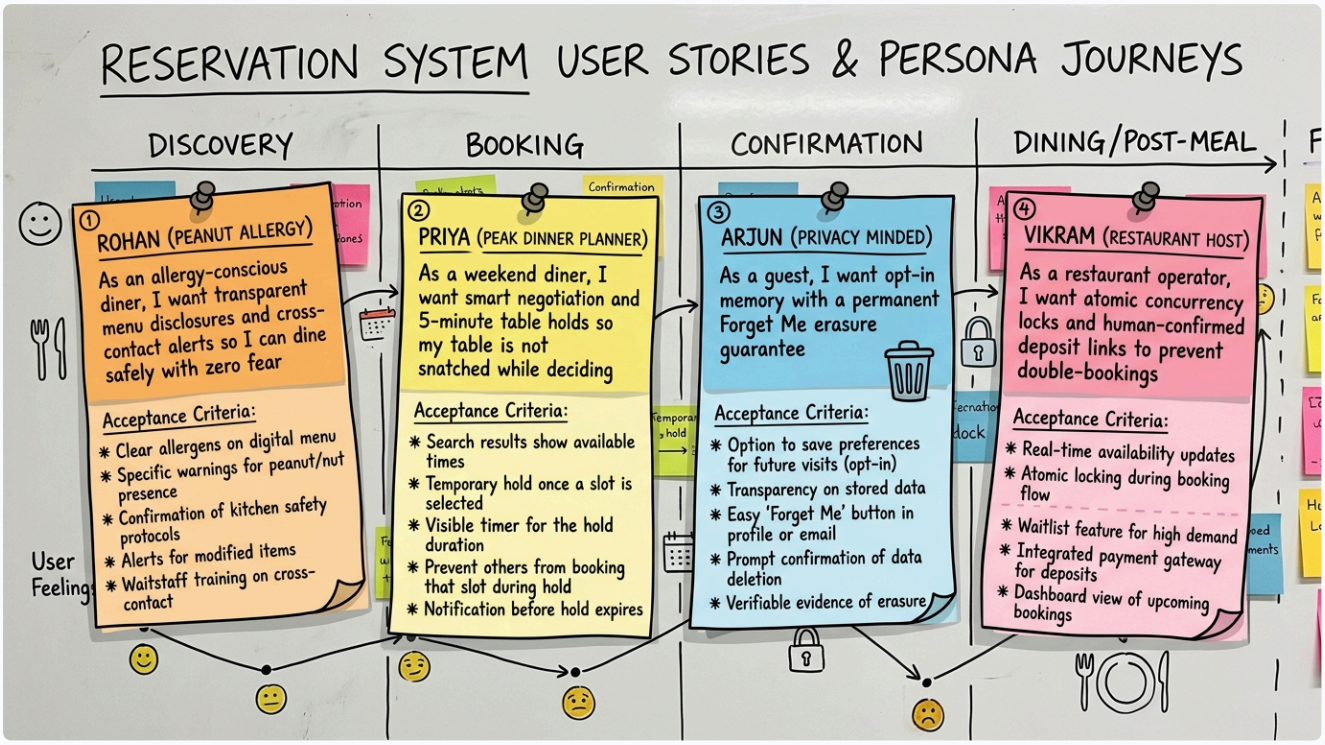


Figure 5: Personas, Agile User Stories, and Acceptance Criteria

Rohan (Allergy-Conscious)

Needs clear disclosures of tree nut and peanut cross-contact so he can order with complete peace of mind.

Priya (Peak Hours Planner)

Needs transparent alternative slot recommendations and short-lived table holds so tables aren't lost while deciding.

Arjun (Privacy Minded)

Requires explicit consent for saved preferences and permanent cryptographic deletion upon request.

Vikram (Restaurant Host)

Relies on serialized concurrency locks to prevent double-bookings and secure deposit links for group bookings.